

SVD for Learning Word Representations

Singular Value Decomposition (SVD) represents one of the most elegant and powerful techniques in linear algebra, offering a sophisticated approach to understanding the underlying structure of data. When applied to natural language processing, SVD transforms the challenge of representing words into a mathematical framework that captures semantic relationships with remarkable precision.

At its core, SVD reveals that any matrix A can be decomposed into three fundamental components: $A = U\Sigma V^T$. This decomposition isn't merely a mathematical curiosity—it's a window into the latent structure of our data, enabling us to extract meaningful patterns from what might otherwise appear as chaotic information.



Understanding SVD: The Mathematical Foundation

Matrix A

The original data matrix containing our term-document relationships. Each element represents the frequency or occurrence of specific terms within documents, creating a sparse but information-rich structure.

Matrix U

Contains orthonormal column vectors representing words in the new latent semantic space. These vectors capture the relationships between terms based on their co-occurrence patterns across documents.

Matrix Σ

A diagonal matrix containing singular values in descending order. These values indicate the importance of each latent dimension, allowing us to rank and potentially discard less significant components.

Matrix V^T

Contains orthonormal row vectors representing documents in the latent space. This matrix reveals how documents relate to each other through their shared semantic content and word usage patterns.

The beauty of SVD lies in its ability to reveal hidden dimensions in our data. Unlike simple word counting, SVD identifies abstract concepts that thread through multiple documents, creating a richer understanding of semantic relationships. This decomposition enables dimensionality reduction whilst preserving the most crucial information, making it invaluable for large-scale text analysis.

Visualising SVD: Geometric Transformation



V^T : Initial Rotation

The first transformation rotates our coordinate system, changing the basis from the original space to align with the principal components of variation in our data. This rotation reveals the natural axes along which our data varies most significantly.



Σ : Scaling Operation

The diagonal matrix stretches the space along each axis by the corresponding singular values. Larger singular values indicate directions of greater variation, whilst smaller values represent less significant dimensions that can often be discarded.



U : Final Rotation

The final transformation rotates the scaled space to its ultimate orientation. This rotation maps our transformed data back into a coordinate system where we can interpret the results in terms of our original problem domain.

This geometric interpretation provides crucial intuition: SVD doesn't merely decompose matrices—it reveals the fundamental geometric structure of our data. In the context of word representations, this means uncovering the semantic dimensions that best explain how words relate to each other across different contexts and documents.

SVD in Natural Language Processing

Why SVD for Word Representations?

Traditional approaches to representing words often struggle with the inherent sparsity and high dimensionality of text data. A typical term-document matrix might contain millions of words across thousands of documents, with most cells containing zero values. This sparsity makes direct analysis challenging and computationally expensive.

SVD addresses these challenges by:

- Reducing dimensionality whilst preserving semantic relationships
- Converting sparse matrices into dense, meaningful representations
- Capturing latent semantic relationships that aren't obvious from surface-level word co-occurrence
- Providing a principled mathematical foundation for Latent Semantic Analysis

The resulting word vectors encode semantic similarity: words used in similar contexts will have similar vector representations, even if they never appear together directly. This property makes SVD-based representations particularly valuable for tasks requiring semantic understanding.



📌 **Key Insight:** SVD transforms the challenge of understanding relationships between thousands of words and documents into a manageable problem in a much lower-dimensional space, typically 50-300 dimensions instead of thousands.

Constructing the Term-Document Matrix

Before applying SVD, we must transform our text collection into a structured mathematical representation. The term-document matrix serves as the foundation for our analysis, where each row represents a unique term and each column represents a document in our collection.

Example: Building Our Matrix

Term	Doc1: "cat mat"	Doc2: "dog yard"	Doc3: "cat dog friends"
cat	1	0	1
mat	1	0	0
dog	0	1	1
yard	0	1	0
friends	0	0	1

This matrix A is 5 terms × 3 documents, where each cell contains the frequency count of a term in a specific document. Notice the sparsity: many cells contain zero, reflecting the reality that most words don't appear in most documents. This simple example demonstrates the fundamental structure that SVD will decompose to reveal latent semantic relationships.

In practice, term-document matrices are often much larger and may incorporate weighting schemes like TF-IDF (Term Frequency-Inverse Document Frequency) to better reflect the importance of terms across the corpus. However, the basic principle remains the same: we're creating a numerical representation of the relationship between words and the contexts in which they appear.

Applying SVD: The Decomposition Process

01

Matrix Decomposition

Apply SVD to decompose our term-document matrix A into three matrices: $A = U\Sigma V^T$. This reveals the fundamental structure underlying our text data.

03

Rank and Truncate

Examine singular values in Σ to identify the most significant dimensions. Truncate to the top k components to achieve dimensionality reduction while preserving essential information.

The decomposition process transforms our sparse, high-dimensional matrix into three dense matrices that capture the essential relationships in our data. The key insight is that we can achieve significant dimensionality reduction—from potentially thousands of dimensions down to just 50-300—whilst retaining the most important semantic information.

The truncation step is crucial: by keeping only the largest singular values and their corresponding components, we eliminate noise whilst preserving the strongest patterns in our data. This not only reduces computational complexity but often improves performance by filtering out less meaningful variations that might otherwise interfere with semantic analysis.

02

Interpret Components

Matrix U contains word vectors in the reduced semantic space, Σ holds singular values indicating dimensional importance, and V^T contains document representations.

04

Generate Embeddings

Multiply U_k and Σ_k to create dense word embeddings. These vectors capture semantic relationships in a compact, computationally efficient representation.

Numerical Example: Step-by-Step SVD Calculation

Let's work through a complete numerical example using our term-document matrix to demonstrate the SVD process in detail.

Given Matrix A:

$$A = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Step 1: Compute $A^T A$

$$A^T A = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

Step 2: Find Eigenvalues

Solving $\det(A^T A - \lambda I) = 0$:

- $\lambda_1 = 3.732$ (largest eigenvalue)
- $\lambda_2 = 2.000$
- $\lambda_3 = 0.268$ (smallest eigenvalue)



Step 3: Calculate Singular Values

$$\sigma_i = \sqrt{\lambda_i}$$

- $\sigma_1 = \sqrt{3.732} = 1.932$
- $\sigma_2 = \sqrt{2.000} = 1.414$
- $\sigma_3 = \sqrt{0.268} = 0.518$

Step 4: Form Σ Matrix

$$\Sigma = \begin{bmatrix} 1.932 & 0 & 0 \\ 0 & 1.414 & 0 \\ 0 & 0 & 0.518 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

For dimensionality reduction, we might choose to keep only the first two components ($k=2$), as the third singular value (0.518) is significantly smaller. This truncation gives us $A_2 = U_2 \Sigma_2 V_2^T$, which approximates our original matrix using only two dimensions instead of three, reducing complexity whilst preserving the most important semantic relationships.

The numerical values demonstrate how SVD quantifies the importance of different dimensions in our data. The largest singular value (1.932) indicates the most significant pattern, whilst the smallest (0.518) represents variation that we might consider noise and choose to discard.

Practical Application: Word Similarity and Semantic Analysis



Example: "Cat" vs "Dog" Similarity

After SVD with $k=2$ dimensions:

- **Cat vector:** [0.58, 0.31]
- **Dog vector:** [0.62, 0.28]

Cosine similarity calculation:

$$\cos(\theta) = \frac{0.58 \times 0.62 + 0.31 \times 0.28}{\sqrt{0.58^2 + 0.31^2} \times \sqrt{0.62^2 + 0.28^2}} = 0.87$$

This high similarity score (0.87) indicates that "cat" and "dog" are semantically related in our corpus, which makes intuitive sense as both are animals that appear in similar contexts.

📌 **Key Insight:** SVD captures latent relationships that go beyond simple co-occurrence, identifying that "cat" and "dog" are similar even though they never appear together in the same document.

Computing Semantic Similarity

Using our reduced word vectors from $U_2\Sigma_2$, we can calculate semantic similarity between words using cosine similarity:

$$\text{similarity} = \frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\| \|\mathbf{v}_2\|}$$

Advantages of SVD Embeddings

- Captures semantic similarity through dimensional reduction
- Provides dense representations from sparse data
- Mathematically principled approach with clear interpretation
- Enables clustering and similarity computations

Practical Applications

- Information retrieval and document search
- Text classification and clustering
- Recommendation systems
- Foundation for advanced NLP techniques

The power of SVD for word representations lies in its ability to discover semantic dimensions that aren't immediately apparent from surface-level analysis. By reducing dimensionality whilst preserving the most important relationships, SVD creates word embeddings that capture meaning in a computationally efficient format, forming the foundation for numerous advances in natural language processing and information retrieval.